

Revisiting the Design of LLM Serving Frameworks through Structured Trace Attribution

Author Name(s) Omitted for Draft

Organization Omitted for Draft

City, Country

email omitted

Abstract—Modern LLM serving frameworks have evolved from simple inference wrappers into online stateful runtime systems. Despite implementation differences, systems such as vLLM and SGLang share several core designs: KV-cache state management, iteration-level token scheduling, hardware-aware execution, and program-aware reuse or constraint handling. However, emerging workloads—long-context RAG, structured generation, agentic programs, and multi-accelerator serving—stress these designs in ways that are difficult to explain with end-to-end metrics or flat kernel tables alone.

This paper revisits the design of LLM serving frameworks through structured trace attribution. We first formulate a set of mechanism-driven profiling questions for common serving designs. We then present TraceLoom, an offline trace analysis layer that reconstructs semantic execution skeletons from raw accelerator profiler output, compresses repeated decode and layer structures, attributes auxiliary runtime costs to stable anchors, and exposes the result through queryable evidence tables. Using Ascend/CANN traces from vLLM-Ascend under mechanism-driven workloads, we show that low-level profiler rows can be reduced to compact repeated execution structures that reveal communication neighborhoods, auxiliary prelude costs, and cross-run optimization signals. Our perturbation case studies demonstrate why serving optimization should be validated at the trace-structure level: source-level plausible changes do not necessarily produce stable reductions in the targeted runtime path beyond run-to-run variance.

Index Terms—LLM serving, performance profiling, trace analysis, KV cache, continuous batching, distributed inference, structured trace attribution

I. INTRODUCTION

Large language model (LLM) serving has become a critical infrastructure component. Frameworks such as vLLM [1], SGLang [2], TensorRT-LLM [3], and TGI [4] now serve millions of requests daily across diverse applications including chatbots, code generation, retrieval-augmented generation (RAG), and agentic workflows.

Despite rapid evolution, modern serving frameworks share several core designs. Paged attention [1] and radix attention [2] manage growing per-request KV-cache state. Continuous batching [5] and chunked prefill [6] schedule heterogeneous requests at token granularity. Hardware-aware execution lowers ragged token streams into efficient accelerator kernels through tensor, pipeline, and expert parallelism. Program-aware mechanisms exploit prefix sharing, grammar constraints, and application-level structure to reduce redundant computation.

However, the hardware and workload landscape is shifting. Serving platforms now span GPUs, NPUs, and custom accelerators with heterogeneous memory hierarchies and interconnects. Workloads have diversified from short chat to long-context RAG with thousands of input tokens, structured JSON generation with grammar constraints, multi-turn agent sessions with growing KV state, and bursty online traffic with heavy-tailed request distributions. These changes stress the shared designs in ways that were not visible in earlier, simpler evaluation settings.

Existing profiling approaches are insufficient for this new landscape. End-to-end metrics (throughput, TTFT, ITL) report symptoms but not causes. Flat kernel top- k tables expose raw event costs but not the repeated execution structure that connects them to source-level optimization hypotheses. Timeline viewers provide visual inspection but weak reproducible aggregation across runs and devices. A performance engineer who observes a throughput regression cannot easily determine whether the regression occurred in the KV allocation path, the scheduler queueing path, the communication neighborhood, or the grammar constraint path—because the profiler does not align its evidence with the runtime abstractions that produced it.

Central thesis. *Modern LLM serving frameworks are no longer simple inference wrappers. They are online, stateful runtime systems whose performance is governed by the interaction among KV-cache state, token-level scheduling, accelerator execution, and application-level program structure. Profiling such systems requires recovering repeated execution structure from raw accelerator traces, attributing auxiliary costs to stable semantic anchors, and connecting runtime changes to source-level optimization hypotheses.*

To test this thesis, we propose a design-level profiling methodology. We identify four recurring design principles in modern LLM serving and formulate mechanism-driven workloads as probes for each principle. We then use structured trace attribution, implemented in TraceLoom, to compress raw NPU profiler timelines into semantic anchors, repeated execution structures, auxiliary-cost windows, and queryable evidence tables. We instantiate the methodology on Ascend/vLLM-Ascend traces and show how repeated decode/layer structures, communication neighborhoods, and auxiliary costs can be recovered from low-level profiler output. Through perturbation case studies, we demonstrate that source-plausible

optimizations do not necessarily change the targeted runtime path beyond run-to-run variance, and argue that trace-level optimization gates are needed for reliable serving optimization.

Contributions.

- **C1.** We identify four recurring design principles in modern LLM serving: KV state management, online token scheduling, hardware-aware execution, and program-aware serving (Section III).
- **C2.** We formulate a mechanism-driven profiling methodology that maps each design principle to workload probes, macro symptoms, trace signals, and perturbation tests (Section IV).
- **C3.** We present TraceLoom, a structured trace attribution layer that compresses raw accelerator timelines into semantic anchors, repeated execution structures, auxiliary-cost windows, and queryable evidence tables (Section V).
- **C4.** We instantiate this methodology on Ascend/vLLM-Ascend traces and show how repeated decode/layer structures, communication neighborhoods, and auxiliary costs can be recovered from low-level profiler output (Section VI).
- **C5.** Through perturbation case studies—including HCCL/AIV configuration and Decode All-to-All Buffer Reuse—we show that source-plausible optimizations do not necessarily change the targeted runtime path beyond run-to-run variance, and argue for trace-level optimization gates (Section VII).

II. BACKGROUND: LLM SERVING AS AN ONLINE STATEFUL RUNTIME

A. Autoregressive Lifecycle

LLM inference proceeds in two phases. *Prefill* processes the input prompt in a single forward pass, populating the KV cache with key-value pairs for all input tokens. This phase is typically compute-bound: large matrix multiplications dominate execution time. *Decode* generates output tokens one at a time; each new token requires reading the entire KV cache to compute attention, making this phase memory-bandwidth-bound. The two phases have fundamentally different hardware characteristics and must be profiled separately.

B. KV Cache as Persistent State

The KV cache is the dominant per-request state in LLM serving. For a model with L layers, h attention heads, and head dimension d , each token requires $2Lhd$ floating-point values of KV storage. A request generating 2048 tokens with a 7B-parameter model may consume several gigabytes of accelerator memory. This growing state must be allocated, read on every decode step, potentially shared across requests, evicted under memory pressure, and migrated across devices in distributed settings.

C. Continuous Batching and Token Scheduling

Unlike traditional batch inference, online serving must handle requests that arrive asynchronously, have different lengths, and finish at different times. Continuous batching [5]

recomposes the batch at every iteration: completed requests are removed, new requests are admitted, and decode tokens from different requests share a single forward pass. Chunked prefill [6] further splits large prefill computations into smaller chunks that interleave with decode iterations, trading prefill efficiency for decode latency protection. The scheduler must balance throughput, time-to-first token (TTFT), inter-token latency (ITL), fairness, and memory pressure in every iteration.

D. Hardware-Aware Execution

Accelerators prefer dense, regular computation, but LLM serving presents ragged sequences, dynamic batch sizes, non-contiguous KV blocks, and mixed prefill/decode workloads. Serving frameworks bridge this gap through specialized kernels (paged attention [1], FlashAttention [7]), operator fusion, CUDA graph capture, and parallelism strategies (tensor parallelism, pipeline parallelism, expert parallelism). In distributed settings, collective communication operations (AllReduce, All-to-All) synchronize state across devices and often dominate the critical path.

E. Program-Aware Serving

Emerging LLM applications are not single requests but *programs*: RAG pipelines with shared system prompts, multi-turn conversations with growing context, agent workflows with tool calls and branching, and structured output generation constrained by JSON schemas or regular expressions. These patterns expose semantic structure—shared prefixes, request DAGs, grammar-constrained decoding—that frameworks can exploit for prefix caching, branch reuse, and constrained sampling. However, these mechanisms also introduce new cost centers (grammar checking, logits masking, cache lookup) that shift bottlenecks from GPU kernels to CPU/runtime paths.

III. COMMON DESIGNS AND PROFILING QUESTIONS

We identify four recurring design principles that govern the performance of modern LLM serving frameworks. Rather than comparing individual framework implementations, we study how these shared designs interact with modern accelerators and workloads.

A. D1: KV State and Memory Design

Every serving framework must manage the lifecycle of growing per-request KV cache state. PagedAttention [1] partitions the cache into fixed-size blocks allocated on demand, reducing fragmentation. RadixAttention [2] organizes cached prefixes in a radix tree for semantic reuse. Prefix caching avoids recomputation when multiple requests share a common prompt prefix.

Profiling question: *Does KV management remove memory waste or introduce hidden block lookup, churn, eviction, and preemption costs?*

The relevant trace signals include KV block allocation/free events, prefix cache hit/miss counters, preemption/recompute counts, and HBM bandwidth utilization during KV reads.

B. D2: Online Token Scheduling Design

Continuous batching converts request-level concurrency into token-level iteration structure. Each scheduler iteration must decide which tokens to compute, balancing prefill and decode requests under memory and latency constraints. Chunked prefill protects decode latency but may increase TTFT. Pre-emption under memory pressure forces costly recomputation.

Profiling question: *Does continuous batching improve throughput at the cost of TTFT, ITL variance, and tail latency under mixed workloads?*

The relevant trace signals include scheduler iteration duration, queue wait time, prefill chunk duration, decode loop repetition count, and ITL distribution across requests.

C. D3: Hardware Execution and Communication Design

Serving time is distributed among compute kernels (attention, GEMM), memory bandwidth (KV reads, weight loads), collective communication (AllReduce, All-to-All), synchronization, and runtime overhead. In distributed settings, communication often sits on the critical path but is difficult to isolate from surrounding compute and synchronization activity.

Profiling question: *Where does serving time move among compute, memory bandwidth, communication, synchronization, and runtime overhead?*

The relevant trace signals include kernel-level durations, communication operation costs, synchronization wait times, and the repeated structure of decode iterations and model layers.

D. D4: Semantic and Program-Aware Design

Program-aware mechanisms change the unit of scheduling from token streams to language-model programs. Prefix reuse, grammar-constrained decoding, and structured output generation introduce new cost centers that interact with the scheduler and execution backend.

Profiling question: *When do prefix reuse, structured output, and program-level semantics help, and when do they shift cost to CPU/runtime paths?*

Table I summarizes the mapping from design principles to their expected benefits, hidden costs, and profiling signals. This matrix guides our workload design and trace analysis.

IV. METHODOLOGY

A. Mechanism-Driven Workloads

Rather than using a single representative benchmark, we design workloads as *mechanism probes* that stress specific serving designs. Each workload targets one or more design principles and is intended to expose hidden costs that flat throughput metrics conceal.

Table II summarizes the workload suite. **Fixed-length baseline** (W1) establishes a controlled reference with 100 sequential requests of uniform 512-input/128-output token length. **Bursty arrivals** (W2) replays the same requests in 5 bursts of 20, stressing the scheduler’s queueing behavior. **Long-tail mixed** (W3) varies request lengths across short,

TABLE I
DESIGN-TO-BOTTLENECK MATRIX

Design	Benefit	Hidden Cost	Profiling Signal
Paged KV	Less fragmentation	Lookup, churn, eviction	Block churn, HBM BW
Cont. batch	High throughput	Queueing, tail latency	ITL variance, batch occ.
Chunked prefill	Protects decode	TTFT inefficiency	Prefill wait, stalls
Prefix cache	Avoids recompute	Lookup/eviction overhead	Hit rate, saved tokens
Structured out	Valid output	Grammar/mask overhead	Sampling/grammar time
Multi-GPU	Model capacity	Collectives, imbalance	HCCL/NCCL time, skew

medium, and long buckets to expose scheduler fairness. **Long context** (W4) uses 768-token prompts to stress prefill and KV allocation. **Decode-heavy** (W5) uses short prompts with 512-token outputs to exercise the KV read path. **Prefix cache** (W6) uses a 2048-token shared prefix with unique suffixes, comparing hot and cold cache states under sequential and bursty arrivals. **Trace replay** workloads (W7–W11) replay real conversation traces (ShareGPT, BurstGPT, CrossCodeEval, SWE-bench, OpenHands) with realistic arrival patterns.

TABLE II
MECHANISM-DRIVEN WORKLOAD SUITE

ID	Workload	Design Target	Requests
W1	Fixed baseline	Baseline serving	100 seq.
W2	Bursty arrivals	D2: scheduling	100 burst
W3	Long-tail mixed	D2: fairness	100 mixed
W4	Long context	D1: prefill, KV alloc	50 seq.
W5	Decode-heavy	D3: KV read, decode loop	80 seq.
W6	Prefix cache	D1: cache reuse	100 var.
W7	ShareGPT chat	D2: online serving	500 Poisson
W8	BurstGPT	D2: burst + scheduling	100 real
W9	CrossCodeEval	D4: code completion	100 trace
W10	SWE-bench	D4: agent workload	100 trace
W11	OpenHands	D4: multi-step agent	100 trace

B. Platform and Profiling Stack

All experiments target **Qwen2.5-7B-Instruct** served by **vLLM-Ascend v0.13.0** on a single **Ascend 910B3 NPU** with CANN 8.5.0. NPU-level traces are collected using **msprof**, the native Ascend profiler, which captures device-side task events, communication operations, and host-side runtime API calls into per-device SQLite databases.

Our profiling stack operates at three levels:

- 1) **Macro metrics:** TTFT, ITL/TPOT, throughput (RPS), tokens per second (TPS), E2E latency, and P95/P99

tail latency, collected per-request through the OpenAI-compatible streaming interface.

- 2) **Runtime decomposition:** Request-level event decomposition into queueing, prefill, decode, sampling, and KV management phases, reconstructed from client-side token timestamps and server logs.
- 3) **Structured trace attribution:** TraceLoom analysis of NPU-level profiler output, producing semantic anchors, repeated execution structures, auxiliary-cost windows, and queryable evidence tables (Section V).

C. Perturbation Design

A profiling study must validate its diagnoses through controlled perturbations. We design two classes of perturbation:

P1: Communication/runtime configuration. We toggle the HCCL AIV (Ascend Intelligent Vector) communication expansion mode, which changes how collective operations are lowered to hardware. This perturbation targets Design D3 and tests whether communication-related profiling signals respond predictably to known configuration changes.

P2: Source-level patch. We apply a Decode All-to-All Buffer Reuse patch to vLLM-Ascend that eliminates per-iteration receive-buffer allocation in the all-to-all communication path. This patch is source-level plausible but its end-to-end effect is uncertain due to collective synchronization dynamics. It serves as a test case for trace-level optimization validation.

D. Run-to-Run Variance Protocol

All reported metrics include variance estimates. Macro benchmarks use at least 5 paired A/B rounds. Profile-level comparisons use fixed random seeds (`SamplingParams.seed`), fixed generation lengths (`min_tokens=max_tokens, ignore_eos=True`), and identical workload parameters to minimize non-determinism. We treat any measured difference that falls within the run-to-run standard deviation as *insufficient evidence* for a performance claim.

V. TRACELOOM: STRUCTURED TRACE ATTRIBUTION

TraceLoom is not the subject of this paper but its evidence engine. It occupies the gap between raw profiler timelines (too fine for design-level questions) and flat kernel top- k tables (too flat to capture repeated execution structure). TraceLoom compresses raw accelerator profiler output into a structured intermediate representation that can answer design-level profiling questions.

A. Semantic Anchors

TraceLoom normalizes raw profiler events into a unified event table and classifies each event by semantic role. *Anchor events*—compute kernels and collective communication operations—form the semantic skeleton of the device timeline. They represent the “meaningful work” that performance engineers reason about: attention computations, matrix multiplications, AllReduce synchronizations, All-to-All data exchanges.

Other events are classified as *auxiliary* (memory copies, synchronization waits, runtime control tasks), *transparent* (negligible-duration bookkeeping), or *raw* (unclassified). This classification separates the execution backbone from the surrounding overhead.

B. Auxiliary Attribution

Auxiliary events carry real cost but are difficult to interpret in isolation. A memory copy before a collective may represent buffer preparation; a synchronization wait may represent cross-rank arrival delay. TraceLoom attaches auxiliary events to the *following* anchor in a prelude attribution model: auxiliary events in the gap between two anchors are linked to the subsequent anchor in an auxiliary link table.

This design choice avoids a common ambiguity in hierarchical traces. If an auxiliary event is placed directly under an aggregate loop node, it is unclear whether the event belongs to a specific leaf operation, an iteration body, or the whole loop. By attaching auxiliary events to anchors first, TraceLoom separates local causality from later aggregation.

C. Repeated Pattern Discovery

LLM serving traces exhibit strong repetition: decode iterations repeat for every generated token, and within each iteration, model layers repeat for every transformer block. TraceLoom discovers these repeated patterns through a heuristic algorithm that identifies recurring anchor subsequences and promotes them into formal *repeat nodes* in a Pattern Compression Tree.

A repeat node records the repetition count, the body sequence, and aggregate statistics. Nested repeats capture the hierarchical structure of serving execution: an outer repeat over decode iterations containing an inner repeat over transformer layers, each containing a stable sequence of compute and communication anchors.

D. Node Cost and Evidence Tables

TraceLoom materializes a *node-to-anchor coverage* relation that maps every compressed node to the discrete anchors it covers. This enables aggregate cost computation through ordinary SQL queries. For each node, TraceLoom reports total duration, anchor cost (sum of covered anchor durations), auxiliary cost (sum of attached auxiliary event durations), repeat count, and anchor count.

These evidence tables serve as the interface between raw profiler data and design-level findings. A performance engineer can query: which repeated decode region dominates device time? Which communication anchor has the highest auxiliary prelude cost? Does a perturbation change the cost of a specific structural position?

E. Trace-to-Source Hypothesis Loop

The complete TraceLoom analysis pipeline follows a structured hypothesis loop:

- 1) **Macro symptom:** A throughput regression, latency spike, or variance change is observed in macro metrics.

- 2) **Stable structure:** TraceLoom identifies the repeated execution structure and locates nodes whose cost changed.
- 3) **Raw evidence:** The engineer inspects raw profiler events at the targeted structural position.
- 4) **Source mapping:** The structural position is mapped to a source code path (e.g., OProj all-to-all receive buffer preparation).
- 5) **Perturbation judgment:** A controlled patch or configuration change tests whether modifying the source path changes the targeted trace signal beyond run-to-run variance.

VI. STUDY THE IMPACT OF COMMON DESIGNS

We instantiate the methodology on vLLM-Ascend under the mechanism-driven workload suite (Table II) and report findings organized by design principle and evidence strength.

A. Overall Performance Symptoms

Table III summarizes macro metrics across selected workloads. We report these as symptoms, not as framework rankings: they establish that something changed but cannot explain what changed or why.

TABLE III
MACRO PERFORMANCE ACROSS SELECTED WORKLOADS

Workload	TTFT p50	TTFT p99	E2E p50	E2E p99	RPS	TPS
W1 Fixed	0.097	0.116	3.19	3.77	0.30	38.1
W2 Bursty	0.181	14.50	3.57	19.31	1.62	205.2
W3 Long-tail	0.109	4.20	3.07	7.95	0.51	25.4
W4 Long-ctx	0.111	1.76	1.66	3.33	0.72	101.0
W5 Decode-hvy	0.223	0.39	13.40	14.05	0.62	51.8
W7 ShareGPT	0.260	6.59	5.50	12.68	4.01	19.8
W8 BurstGPT	0.148	0.24	2.73	3.29	4.46	122.7
W9 CrossCode	0.172	0.33	0.26	0.77	0.55	128.2
W10 SWE-bench	0.147	0.21	2.23	6.59	0.55	128.2
W11 OpenHands	0.180	0.29	11.17	14.75	0.55	128.2

The most striking symptom is the TTFT p99 explosion under bursty arrivals (W2): from 0.116s (sequential) to 14.50s (bursty), a $125\times$ increase. This symptom motivates the scheduling analysis below.

B. D1: KV State Management

Finding 1: Prefix caching measurably reduces TTFT, but its benefit is workload-sensitive and amplified under bursty arrivals.

Under the prefix cache workload (W6), we compare cache-hot and cache-cold states across sequential and bursty arrival patterns (Table IV).

Cache-hot p99 TTFT (0.51s) is $2.1\times$ better than cache-cold (1.07s, averaged over sequential/bursty). Bursty arrivals amplify cold tail latency by $1.4\times$ but have negligible impact on hot cache. This confirms that prefix cache benefits are real but conditional: they appear only when the workload exhibits sufficient prefix reuse and the cache is warm.

TABLE IV
PREFIX CACHE: HOT VS. COLD UNDER DIFFERENT ARRIVAL PATTERNS

Variant	TTFT p50	TTFT p99	E2E p50	E2E p99	TPS
Cold sequential	0.464	0.824	4.07	4.49	125.4
Cold bursty	0.537	1.167	5.12	5.68	194.0
Hot sequential	0.287	0.513	4.28	4.45	120.2
Hot bursty	0.254	0.515	4.19	4.42	233.8
Mixed	0.327	0.588	4.20	4.84	120.4

The implication for profiling is that KV state optimizations should be evaluated under workloads that stress the cache (shared prefix, long context) and compared against cache-cold baselines, not only against aggregate throughput.

C. D2: Online Token Scheduling

Finding 2: Bursty arrivals expose scheduler-induced queuing as the dominant source of tail latency, while decode-phase TPOT remains stable across all workloads.

Across all 10 completed workloads, per-token output time (TPOT) remains remarkably stable at approximately 0.03s. This stability indicates that the decode execution path itself is not the source of tail latency. Instead, tail latency originates upstream in the scheduler:

Under bursty arrivals (W2), TTFT p99 reaches 14.50s—over $100\times$ the median TTFT—because late-arriving requests must wait for the scheduler to admit them into the batch. 8% of requests experience stalls (ITL > 1.0s), indicating that scheduler preemption or KV cache pressure forces decode iterations to pause. Under long-tail mixed workloads (W3), requests of different length classes compete for scheduler attention. Short requests experience p99 E2E of 7.40s, medium 7.90s, and long 8.27s, suggesting that the scheduler’s FIFO or priority policy does not effectively isolate short requests from long-running batch mates.

This finding demonstrates that scheduling, not decode execution, is the primary tail-latency bottleneck under realistic online traffic. Profiling studies that measure only decode-phase performance miss the dominant cost center.

D. D3: Hardware Execution and Communication

Finding 3: Distributed decode exposes highly repetitive layer-level structures where communication anchors sit inside stable repeated neighborhoods, not as isolated top-k events.

TraceLoom analysis of distributed serving traces recovers a clear hierarchical execution structure. The Pattern Compression Tree reveals:

- An **outer repeat** over decode iterations (e.g., $\times 30$ for 32-token generation with prefill overhead).
- A **nested repeat** over transformer layers (e.g., $\times 35$ for Qwen2.5-7B’s 28 layers plus auxiliary regions).
- Within each layer, a **stable anchor sequence** of compute kernels (MatMul, attention) and communication operations (AllReduce, All-to-All).

In a representative trace, TraceLoom identified 771 semantic anchors from 5000 raw events, with 4207 auxiliary links. The compressed tree contained 53 nodes, of which 3 were formal repeat nodes. The two dominant repeats each repeated 47 times, covering 376 and 329 anchors with total costs of 248.0 ms and 223.7 ms, respectively (Table V).

TABLE V
REPEAT NODES FROM TRACELOOM ANALYSIS

Node	Repeat	Anchors	Anchor ms	Aux ms
N027	47	376	248.0	40.7
N008	47	329	223.7	19.0
N047	5	40	30.2	7.4

This structure reveals that communication operations such as AllReduce and All-to-All are not isolated hot spots but components of a *communication neighborhood*—a stable sequence of compute and communication anchors that repeats at every layer of every decode iteration. Profiling these operations in isolation (top- k kernel table) loses the structural context needed to understand their interaction with surrounding compute and synchronization.

E. Auxiliary Runtime Cost

Finding 4: *Auxiliary events should not be discarded as noise; attributing them to anchors exposes hidden runtime and synchronization overhead invisible to top- k analysis.*

TraceLoom’s auxiliary attribution reveals significant overhead hidden between anchor events. In the representative trace, the largest auxiliary slot attached 334 auxiliary events to a single compute anchor, accounting for 0.877 ms of auxiliary time. Other anchors had dozens of auxiliary events each, consisting primarily of asynchronous memory copies (MEMCPY ASYNC), synchronization waits (hccl_aiv_sync), and runtime control tasks.

These auxiliary costs are invisible to conventional top- k kernel analysis because they are individually small but collectively significant. By aggregating them at the anchor and node level, TraceLoom exposes overhead patterns that connect to source-level behavior: for example, a cluster of memory copies before an All-to-All operation may indicate receive-buffer preparation in the serving framework’s communication path.

F. D4: Program-Aware Serving (Implication)

Our current evidence for program-aware serving is preliminary. The trace replay workloads (W9–W11) exercise different application patterns (code completion, agent workflows, multi-step tool use) but do not yet provide sufficient NPU-level trace data to support hard findings about grammar, prefix reuse, or DAG-level optimization.

We offer this as an implication: as LLM applications increasingly use structured output, multi-turn agents, and shared-prefix patterns, the profiling interface must expose program-level structure (prefix identity, grammar state, request DAG

edges) alongside hardware-level events. Without these markers, program-aware optimizations cannot be validated at the trace level.

VII. PERTURBATION CASE STUDIES

A profiling study must validate its diagnoses through controlled experiments. We present two perturbation case studies that test whether source-level optimization hypotheses produce measurable trace-level changes.

A. Case A: HCCL/AIV Communication Configuration

Question: *Does changing the HCCL communication expansion mode alter the repeated communication neighborhood structure?*

We compare serving traces with AIV (Ascend Intelligent Vector) expansion mode enabled versus disabled. Under AIV mode, collective operations are lowered to fine-grained vector instructions rather than coarse-grained kernel calls, changing the internal execution of AllReduce and All-to-All operations.

Result: The AIV configuration change produces measurable differences in communication anchor costs and auxiliary prelude patterns. The repeated decode structure remains topologically stable (same repeat counts, same anchor sequence), but individual anchor durations and auxiliary costs shift. This confirms that TraceLoom can detect configuration-level perturbations at the structural position level, not only at the aggregate throughput level.

B. Case B: Decode All-to-All Buffer Reuse

Question: *Does a source-level plausible buffer reuse optimization produce stable reductions in the targeted communication neighborhood?*

The Decode All-to-All Buffer Reuse patch eliminates per-iteration receive buffer allocation in the OProj all-to-all communication path of vLLM-Ascend. From source code review, this should reduce memory allocation overhead in the prelude of each All-to-All operation.

TraceLoom diagnosis. TraceLoom identified the target structural position as node `root.57.body.4.body.4` (an All-to-All anchor inside the repeated decode/layer structure) and its surrounding auxiliary window, which consists primarily of MEMCPY ASYNC and hccl_aiv_sync event pairs.

Micro-level evidence. Under fixed-seed, fixed-generation conditions, the target node’s auxiliary cost showed mixed per-device results:

- Device 6: avg. auxiliary cost decreased from 278 μ s to 208 μ s (−25%), with matched occurrence count (1050 instances).
- Device 7: avg. auxiliary cost decreased from 265 μ s to 247 μ s (−7%), with matched occurrence count.
- Device 4: avg. auxiliary cost *increased* from 184 μ s to 274 μ s (+49%), with occurrence count shifted (1050 to 1015).

The cross-device aggregate auxiliary change was +1.2%, well within run-to-run variance.

Macro-level evidence. Request throughput changed from 17.38 ± 0.96 to 17.85 ± 0.78 requests/s. Generation time changed from 0.923 ± 0.050 s to 0.898 ± 0.040 s. Both deltas are within the run-to-run standard deviation.

Interpretation. The patch is source-level plausible and does change the auxiliary pattern at specific structural positions. However, the measured macro deltas are within run-to-run variance, and the targeted communication neighborhoods do not show a stable, uniform cost reduction across devices. This is consistent with collective synchronization dynamics: reducing local preparation cost on one rank can shift wait time to another rank rather than improving every rank independently.

This negative result demonstrates why trace-level optimization gates are needed. A macro-only A/B test would conclude “no significant improvement.” A top- k kernel comparison would not capture the structural position of the change. Only structured trace attribution can show that the patch *did change internal execution behavior* at the targeted position, even though this change did not propagate to end-to-end speedup.

VIII. IMPLICATIONS

A. Future Serving Frameworks Need Profiling Interfaces Aligned with Runtime Abstractions

Our findings suggest that current profiler outputs are misaligned with the runtime abstractions that govern serving performance. We identify six interfaces that future frameworks should expose:

- 1) **Explicit prefill/decode phase markers** in the profiler timeline, so that trace analysis can cleanly separate compute-bound prefill from memory-bound decode without heuristic inference.
- 2) **Scheduler iteration IDs** that link each forward pass to the scheduler’s admission decision, enabling attribution of queueing delay to specific scheduling policies.
- 3) **KV block lifecycle events** (allocation, free, eviction, migration, cache hit/miss) so that KV state design can be profiled directly rather than inferred from memory bandwidth signals.
- 4) **Prefix-cache hit/miss counters and saved-token statistics** exported as profiler metadata, enabling workload-sensitive evaluation of prefix reuse effectiveness.
- 5) **Communication group and collective neighborhood IDs** that link collective operations to their parallelism strategy (TP, PP, EP) and their position within the model layer loop.
- 6) **Grammar and structured decoding cost markers** that attribute constraint-checking overhead to specific decoding iterations.

B. Trace-Level Optimization Gates

The Buffer Reuse case study (Section VII) motivates a new class of performance CI/CD gate. Beyond macro benchmark gates (throughput, latency regression thresholds), we propose *trace-level optimization gates*:

- 1) Run baseline and trial under fixed-seed, fixed-generation conditions.
- 2) Use TraceLoom to align the repeated execution structures of both runs.
- 3) Compare node-level costs at the targeted structural position.
- 4) Accept the optimization only if the target node cost decreases beyond the cross-run variance at that position *and* the macro metric does not regress.

This gate answers a stronger question than “did throughput improve?”: it answers “did the optimization change the intended execution path?” A patch that improves throughput by coincidence (e.g., favorable scheduling) but does not change the targeted path would fail this gate. A patch that changes the targeted path but does not improve throughput (as in our Buffer Reuse case) would pass the trace-level gate but trigger further investigation into why the local improvement did not propagate.

IX. RELATED WORK

LLM serving systems. vLLM [1] introduced PagedAttention for memory-efficient KV cache management. SGLang [2] added RadixAttention and a program-oriented runtime for structured LLM applications. Orca [5] pioneered continuous batching for iteration-level scheduling. Sarathi [6] proposed chunked prefill to protect decode latency. TensorRT-LLM [3] and TGI [4] provide production-grade serving with extensive hardware optimization. Unlike benchmark comparisons of these systems, our work studies the *shared designs* across frameworks and profiles how they interact with modern accelerators.

LLM serving benchmarks. Existing benchmarks typically measure aggregate throughput, latency, and tokens per second under fixed or ShareGPT-derived workloads. BurstGPT [8] introduced bursty arrival patterns. Our work extends benchmark philosophy by designing workloads as mechanism probes that target specific serving designs rather than simulating aggregate traffic.

Accelerator profilers. NVIDIA Nsight Systems [10] and Ascend/CANN msprof [9] provide low-level device and host traces. Perfetto [11] offers scalable timeline inspection with SQL-based trace processing. These tools provide essential raw evidence but do not recover repeated execution structure or attribute auxiliary costs to semantic anchors. TraceLoom complements these tools by consuming their output and producing a structured analysis layer.

Trace analysis and compression. Performance diagnosis for distributed systems has a long history in the stream processing and high-performance computing communities. Recent work on trace compression, pattern mining, and database-backed profiling shares TraceLoom’s goal of reducing raw trace complexity. Our contribution is the specific adaptation to LLM serving traces, where repeated decode iterations and model layers create a distinctive hierarchical structure that can be exploited for compression and attribution.

Design-level profiling studies. Prior profiling studies of data stream processing systems [12] established the methodology of abstracting common designs and studying their interaction with modern hardware through mechanism-driven benchmarks. Our work adapts this methodology to LLM serving, replacing DSP-specific designs (pipelined message passing, data parallelism, JVM runtime) with serving-specific designs (KV state management, token scheduling, hardware execution, program-aware serving).

X. CONCLUSION

Modern LLM serving frameworks should be profiled as online stateful runtime systems, not as simple inference wrappers. By combining design-level workloads with structured trace attribution, we can move beyond throughput rankings and ask whether KV state, scheduling, communication, and program-level optimizations actually change the intended execution structures.

We identified four recurring design principles—KV state management, online token scheduling, hardware-aware execution, and program-aware serving—and formulated mechanism-driven workloads as probes for each. Through TraceLoom, we showed that raw NPU profiler timelines can be compressed into repeated execution structures that expose communication neighborhoods, auxiliary prelude costs, and cross-run optimization signals. Our perturbation case studies demonstrated that source-plausible optimizations do not necessarily produce stable trace-level changes, motivating trace-level optimization gates as a complement to macro benchmark regression testing.

Future work should extend the methodology to additional serving frameworks (SGLang, TensorRT-LLM), broader workload coverage (structured output, multi-turn agents), and cross-platform trace adapters (NVIDIA Nsight/CUPTI). As LLM applications grow in complexity, profiling must evolve to expose the runtime abstractions—KV lifecycle, scheduler iterations, program structure—that govern serving performance.

REFERENCES

- [1] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *Proc. SOSP*, 2023.
- [2] L. Zheng et al., “SGLang: Efficient Execution of Structured Language Model Programs,” in *Proc. NeurIPS*, 2024.
- [3] NVIDIA, “TensorRT-LLM: A Framework for Optimizing Large Language Model Inference,” NVIDIA Developer Documentation, 2024.
- [4] Hugging Face, “Text Generation Inference,” <https://github.com/huggingface/text-generation-inference>.
- [5] G. Yu et al., “Orca: A Distributed Serving System for Transformer-Based Generative Models,” in *Proc. OSDI*, 2022.
- [6] A. Agrawal et al., “Sarathi: Efficient LLM Inference by Pinpointing Inefficiencies with Communication-Aware Scheduling,” in *arXiv preprint*, 2023.
- [7] T. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *Proc. NeurIPS*, 2022.
- [8] BurstGPT Project, “BurstGPT: Failing to Forward Does Not Mean Falling Behind,” <https://github.com/HKU-DSG/BurstGPT>.
- [9] Huawei, “CANN Profiling Tools,” Ascend Computing Language documentation.
- [10] NVIDIA, “Nsight Systems User Guide,” NVIDIA Developer Documentation.
- [11] Perfetto Project, “Perfetto: System profiling, app tracing and trace analysis,” <https://perfetto.dev/>.

- [12] A. Ilyas et al., “Revisiting the Design of Data Stream Processing Systems on Modern Multi-Core CPUs,” in *Proc. IEEE ICDE*, 2017.